Lab Bài 6: Mini RAG với LangChain/LlamaIndex – hỏi đáp sổ tay vận hành

Brief ngắn để sinh notebook Google Colab

Lab handout – 1,5 giờ

Mục tiêu Lab

- Tạo notebook Mini RAG dùng index từ Bài 5 hoặc tạo lại nhanh.
- Dùng **LangChain hoặc LlamaIndex** ghép retrieval, grounded prompt và LLM.
- Yêu cầu câu trả lời có nguồn và biết nói không đủ thông tin.
- Chạy bộ 10 câu hỏi benchmark và ghi error analysis (faithfulness/relevancy/context recall).
- Thử 2 cấu hình chunk/top-k/prompt và so sánh kết quả.

Input cần chuẩn bị

- 10–30 tài liệu/sổ tay ngắn về vận hành mạng hoặc IT operations.
- Mỗi tài liệu có id, title, section, version/date và nội dung text.
- 10 câu hỏi benchmark: có đáp án rõ, cần nhiều chunk, mơ hồ, không có đáp án.
- Expected evidence cho từng câu hỏi, ví dụ doc id + section.
- Tùy chọn: index FAISS/Chroma đã tạo từ Lab Bài 5.

Cấu trúc notebook đề xuất

- 1. Business context và use case.
- 2. Load documents và benchmark questions.
- 3. Chunking + embedding + vector index (hoặc load index có sẵn).
- 4. Retriever (qua LangChain/LlamaIndex): input question, output top-k chunks.
- 5. Grounded prompt template.
- 6. RAG answer function (query engine / chain).
- 7. Evaluation table và failure analysis.

Business context

- Người dùng: kỹ thuật viên hoặc điều phối vận hành.
- Pain point: phải tra nhiều sổ tay/checklist khi xử lý incident.
- AI task: trả lời câu hỏi theo tài liệu nội bộ, kèm nguồn.
- Không làm: tự động thực thi hành động hoặc thay đổi cấu hình.
- Mức tự động hóa: Assist hoặc Approve, chưa Automate.

Dùng framework LangChain/LlamaIndex

- Dùng loader/splitter/embedding/vectorstore/retriever dựng sẵn của framework.
- Ghép thành query engine (LlamaIndex) hoặc retrieval chain (LangChain).
- **Luôn mở ra để in top-k chunk** trước khi gọi LLM, đừng chỉ tin câu trả lời cuối.
- Giữ `prompt_version` để log khi thử nghiệm.

Cell nhóm 1 – Indexing

- Load documents từ JSON/CSV/list Python.
- Chia chunk theo cấu hình A: chunk size và overlap mặc định.
- Tạo embedding bằng model được chuẩn bị sẵn.
- Tạo FAISS hoặc Chroma index (qua framework).
- In một vài chunk để học viên kiểm tra bằng mắt.

Cell nhóm 2 – Retrieval

- Viết/khởi tạo retriever(question, top_k).
- Trả về chunk text, doc id, section, score và metadata.
- Hiển thị top-k chunk trước khi gọi LLM.
- Ghi nhận câu hỏi nào lấy sai chunk.
- Tùy chọn: so sánh top-k = 3 và top-k = 5.

Cell nhóm 3 – Grounded prompt

- Template yêu cầu chỉ trả lời từ context.
- Nếu context không đủ, trả lời rõ "không đủ thông tin".
- Output gồm answer ngắn, sources và notes nếu có mâu thuẫn.
- Không cho model dùng kiến thức ngoài tài liệu.

Cell nhóm 4 – RAG answer function

- Input: question, top_k, prompt_version.
- Bước 1: retrieve top-k chunks.
- Bước 2: đóng gói context có source id.
- Bước 3: gọi LLM với grounded prompt.
- Bước 4: trả answer, sources, retrieved chunks và raw response để debug.

Benchmark 10 câu hỏi

- 3 câu hỏi có đáp án trực tiếp trong một chunk.
- 2 câu hỏi cần tổng hợp nhiều chunk.
- 2 câu hỏi có thuật ngữ khác cách viết trong tài liệu.
- 2 câu hỏi không có đáp án trong tài liệu.
- 1 câu hỏi có nguy cơ nhầm phiên bản hoặc nhầm loại thiết bị.

Evaluation table

- question_id, question, expected_evidence.
- retrieved_evidence: đúng/sai/một phần (context recall).
- answer_relevant: 0/1/2.
- faithful: có/không.
- citation_ok: có/không.
- failure_type và note cải thiện.

Experiment so sánh cấu hình

- Cấu hình A: chunk/top-k mặc định.
- Cấu hình B: thay chunk size hoặc top-k, không thay quá nhiều biến cùng lúc.
- Chạy cùng 10 câu hỏi benchmark.
- So sánh số câu retrieve đúng, answer đúng, citation đúng.
- Ghi trade-off latency/token cost nếu có.

Deliverable cuối Lab

- Notebook chạy được từ câu hỏi đến answer có nguồn (LangChain/LlamaIndex).
- Bảng test 10 câu hỏi với faithfulness/relevancy/context recall.
- Ít nhất 3 failure case và phân loại lỗi.
- Kết luận cấu hình nào tốt hơn và vì sao.
- Đề xuất cải thiện: chunking, metadata, prompt, filter, reranking hoặc tài liệu.

Prompt để sinh notebook về sau

- Tạo notebook Colab cho Lab Mini RAG bằng Python, dùng LangChain hoặc LlamaIndex.
- Dữ liệu là danh sách tài liệu vận hành mô phỏng tiếng Việt, có metadata.
- Dùng sentence-transformers + FAISS hoặc Chroma cho embedding/vector search.
- Tạo grounded prompt và hàm/engine rag_answer().
- Bao gồm benchmark 10 câu, bảng evaluation (faithfulness/relevancy/context recall) và phần failure analysis.

Debrief

- RAG sai vì không tìm đúng tài liệu hay vì LLM tổng hợp sai?
- Khi không có bằng chứng, hệ thống có dám nói không đủ thông tin không?
- Nguồn/citation có thực sự hỗ trợ câu trả lời không?
- Nếu triển khai thật, ai chịu trách nhiệm khi câu trả lời sai?
- Có cần RAG hay search/rule đơn giản đã đủ?